Experiment 1: Selection Sort – Trophy Ranking System

Aim

To implement the **Selection Sort** algorithm in Python to find the **Top 5 highest scores** from a list of participants without sorting the entire array.

Problem Statement

A sports event organizer has recorded the scores of **50 participants** but needs to display **only the Top 5 scores** on the leaderboard. Instead of sorting the complete array, use Selection Sort's **find maximum repeatedly** approach to extract the Top 5 scores.

Algorithm

Step 1: Start.

Step 2: Read the list of scores.

Step 3: Read the value of **k** (number of top scores required).

Step 4: Repeat **k** times:

- Find the maximum element in the remaining array.
- Swap it with the current position.

Step 5: Print the first **k** elements as the Top Scores.

Step 6: Stop.

Source Code (Python)

main.py

```python
# Selection Sort - Top K Scores

scores = list(map(int, input("Enter scores: ").split()))
k = int(input("Enter Top K value: "))

n = len(scores)

for i in range(min(k, n)):
    max_index = i

    for j in range(i + 1, n):
        if scores[j] > scores[max_index]:
            max_index = j

    scores[i], scores[max_index] = scores[max_index], scores[i]

print("Top", k, "Scores:", scores[:k])
```

Output

```
Output
Enter scores: 72 88 65 90 77 95 60 83 91 68
Enter Top K value: 5
Top 5 Scores: [95, 91, 90, 88, 83]

=== Code Execution Successful ===
```

Time Complexity

- **Time Complexity: O(k × n)**
- **Space Complexity: O(1)**

Result

The **Selection Sort** algorithm was successfully implemented in Python. The program repeatedly selects the maximum element and displays the **Top K scores** without sorting the entire array. This approach is efficient when only a few highest values are required.